

Machine Learning Project Based Learning (PBL) Report on
Crop Disease Detection and Fertilizer Recommendation System

Submitted in partial fulfillment of the
Requirements for the award of the degree
of BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering (AI&ML)

Submitted by

M.V.Abhiram

23R11A66P3

Under the esteemed guidance of

Mr. Shaik Akbar

Associate Professor, CSE(AI&ML) Department
Geethanjali College of Engineering and
Technology



Geethanjali College of Engineering and Technology
Department of Computer Science and Engineering(AI&ML)
(UGC AUTONOMOUS INSTITUTION)

Accredited by NAAC with 'A+' Grade, Approved by AICTE and Affiliated to JNTUH Cheeryal

(V), Keesara (M), Medchal (Dist), Telangana – 501 301.

April - 2026

Geethanjali College of Engineering and Technology
Department of Computer Science and Engineering (AI&ML)
(UGC AUTONOMOUS INSTITUTION)

Accredited by NAAC with 'A+' Grade, Approved by AICTE and Affiliated to JNTUH Cheeryal (V),
Keesara (M), Medchal (Dist), Telangana – 501 301.

April – 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)

CERTIFICATE

This is to certify that the Machine Learning Project Based Learning (PBL) Report entitled “Crop Disease Detection and Fertilizer Recommendation System” is a bonafide work done and submitted by M V Abhiram(23R11A66P3) during the academic year 2025 – 2026, in partial fulfillment of the requirement for the award of Bachelor of Technology degree in “Computer Science and Engineering (AI&ML)” from Geethanjali College of Engineering and Technology (Accredited by NAAC with 'A+' Grade, Approved by AICTE and Affiliated to JNTUH), is a bonafide record of work carried out by them under guidance and supervision.

Certified further that to the best of my knowledge, the work in this project has not been submitted to any other institution for the award of any degree or diploma.

FACULTY GUIDE

Mr. Shaik Akbar

Associate Professor

CSE(AI&ML) Department

HEAD OF THE DEPARTMENT

Dr. A. Nageshwara Rao

HOD CSE (AI&ML) Department

DECLARATION

I hereby declare that the project report entitled "Crop Disease Detection and Fertilizer Recommendation System" is an original work done and submitted to the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Geethanjali College of Engineering and Technology, in partial fulfillment of the requirement for the award of Bachelor of Technology in **Computer Science and Engineering (AI&ML)**.

I further declare that this project is a record of bonafide work carried out by me under the guidance of Mr. Shaik Akbar, **Assistant/Associate Professor, Department of CSE (AI&ML)**.

I also declare that the work presented in this project report has not been submitted, either in part or full, for the award of any other degree or diploma in this institute or any other institute/university.

Signature of the student

M V Abhiram

(23R11A66P3)

ACKNOWLEDGEMENT

The satisfaction of completing this project would be incomplete without expressing our sincere gratitude to all the individuals who supported and guided us throughout the project development process.

First and foremost, we would like to thank the **Chairman, Principal, and Vice Principal** of Geethanjali College of Engineering and Technology for providing us with the required infrastructure and facilities to successfully complete this project.

We express our heartfelt gratitude to our respected faculty guide Mr. Shaik Akbar, **Assistant/Associate Professor, Department of CSE (AI&ML)**, for his valuable guidance, continuous encouragement, constructive suggestions, and constant support throughout the development of this project. His technical expertise and motivation helped us complete the project successfully.

We would also like to extend our sincere thanks to the **Head of the Department, Dr. A. Nageshwara Rao**, for his encouragement and support throughout the course of this work.

We are grateful to all the faculty members of the Department of CSE (AI&ML) for their valuable suggestions and assistance whenever required.

Finally, we would like to thank our friends and family members for their encouragement, support, and motivation, which inspired us to complete this project successfully.

ABSTRACT

The agricultural sector faces critical challenges in crop disease management and fertilizer optimization, with crop diseases causing 20-40% of global yield losses annually. Traditional disease diagnosis methods require expert knowledge and laboratory testing, which are often inaccessible to farmers, particularly in remote areas. To address these challenges, this project presents Agri-Smart: An AI-Powered Crop Disease Detection and Fertilizer Recommendation System that provides instant, accurate disease identification and personalized fertilizer recommendations.

The system employs a two-stage artificial intelligence pipeline. The first stage utilizes a Convolutional Neural Network (CNN) trained on 61,955 images across 59 crop-disease combinations, achieving 87.79% validation accuracy in detecting diseases and identifying specific disease types such as Bacterial Blight, Leaf Blast, and Curl Virus. The second stage implements an AdaBoost classifier trained on 10,000 samples, achieving 77.5% accuracy in recommending optimal fertilizers (Urea, DAP, MOP, NPK, SSP, Compost, or Zinc Sulphate) based on crop type, soil characteristics, and environmental factors including Nitrogen, Phosphorus, Potassium levels, Temperature, Humidity, and Soil Moisture.

The system features a user-friendly Streamlit-based web interface where farmers can upload leaf images and receive instant disease diagnosis with gradient-based visualization heatmaps. To enhance transparency and build trust, the system integrates SHAP (SHapley Additive exPlanations) analysis, providing clear explanations of which factors influenced the fertilizer recommendation. This project demonstrates a practical, accessible solution for improving crop management, reducing agricultural losses, and promoting sustainable farming practices through the integration of deep learning, machine learning, and explainable AI.

TABLE OF CONTENTS

INTRODUCTION	1-5
AIM AND OBJECTIVES	6
LITERATURE REVIEW	7-8
SOFTWARE REQUIREMENTS SPECIFICATION	9-10
DESIGN	11-15
DATASETS AND PREPROCESSING	16-18
IMPLEMENTATION	19-23
TRAINING AND TESTING	24-25
SOURCE CODE	26-37
RESULT	38-41
CONLCLUSION	42
FUTURE SCOPE	43
REFERENCES	44

1. INTRODUCTION

Agriculture remains the backbone of global food security, supporting billions of people worldwide. However, the sector faces critical challenges in crop disease management and fertilizer optimization. Crop diseases alone cause 20-40% of global yield losses annually, resulting in billions of dollars in economic damage. Early detection and accurate diagnosis are crucial, yet traditional methods require expert knowledge, laboratory testing, and significant time — resources often unavailable to farmers, especially in rural areas. Farmers typically rely on visual inspection and personal experience, leading to misdiagnosis, delayed treatment, and ineffective disease management.

Compounding this challenge is the complexity of fertilizer management. Determining the right fertilizer requires understanding crop type, soil characteristics, nutrient levels (Nitrogen, Phosphorus, Potassium), and environmental conditions. Most farmers lack access to soil testing facilities and expert advice, leading to improper fertilizer usage. Over-application results in increased costs and environmental damage, while under-application leads to reduced productivity.

Artificial Intelligence (AI) and Machine Learning (ML) offer powerful solutions for transforming agriculture through intelligent automation. Deep Learning, particularly Convolutional Neural Networks (CNN), excels in image classification tasks, making it ideal for automated crop disease detection. Similarly, machine learning algorithms can analyze agricultural parameters to provide data-driven fertilizer recommendations.

This project develops Agri-Smart: Crop Disease Detection and Fertilizer Recommendation System that empowers farmers with instant, accurate insights. The system employs a two-stage pipeline: a CNN trained on 61,955 images across 59 crop-disease combinations achieving 83.45% accuracy for disease detection, and an AdaBoost classifier trained on 10,000 samples achieving 78.5% accuracy for fertilizer recommendation. The system identifies specific disease types (Bacterial Blight, Leaf Blast, Curl Virus, Early Blight, Powdery Mildew, Rust) and recommends optimal fertilizers (Urea, DAP, MOP, NPK, SSP, Compost, Zinc Sulphate) based on crop type, soil characteristics, and environmental factors.

To ensure transparency, the system integrates SHAP (SHapley Additive exPlanations) analysis, explaining which factors influenced recommendations. The Streamlit-based web interface allows farmers to upload leaf images, receive instant disease diagnosis with visualization heatmaps, select soil type, and obtain personalized fertilizer recommendations with feature importance charts.

This project demonstrates the practical application of Deep Learning, Machine Learning, Computer Vision, and Explainable AI to solve real-world agricultural problems, providing farmers with accessible tools for improved crop management, reduced losses, and sustainable farming practices.

1.1 Crop Disease Detection

Crop Disease Detection refers to the process of identifying and diagnosing plant diseases by analyzing visual symptoms and characteristics exhibited by affected crops. It is a specialized application of Deep Learning, and Artificial Intelligence, designed to automatically analyze leaf images and provide accurate disease identification from visual data.

In agriculture, crop diseases pose significant threats to food security and farmer livelihoods. Plant diseases caused by bacteria, fungi, viruses, and pests can rapidly spread across fields, causing severe damage if not detected early. Farmers traditionally rely on visual inspection and personal experience, but this approach is often unreliable, time-consuming, and requires expert knowledge that many small-scale farmers lack.

Importance of Crop Disease Detection:

- 1) Early Disease Identification - Detects diseases in early stages before symptoms become severe, enabling timely intervention.
- 2) Reduced Crop Losses - Early detection helps farmers apply targeted treatments quickly, reducing yield losses that can reach 20-40%.
- 3) Accessibility for All Farmers - Makes expert-level diagnosis accessible to farmers in remote areas lacking agricultural extension services.
- 4) Faster Decision Making - Provides instant disease identification without waiting for expert consultations.
- 5) Sustainable Agriculture - Enables precise pesticide application only when necessary, reducing chemical usage and environmental impact.

1.2 Fertilizer Recommendation

Fertilizer Recommendation refers to the process of determining the optimal type and amount of fertilizer required for specific crops based on soil characteristics, nutrient levels, and environmental conditions. It is a critical aspect of precision agriculture that directly impacts crop productivity, soil health, and farming sustainability.

Proper fertilizer application is essential for maintaining soil fertility and maximizing crop yields. However, determining the right fertilizer requires understanding multiple factors including crop nutrient requirements, soil type, existing nutrient levels (Nitrogen, Phosphorus, Potassium), temperature, humidity, and soil moisture. Most farmers lack access to soil testing facilities and expert agronomic advice, leading to improper fertilizer usage. Over-application results in increased costs, soil degradation, water pollution, and environmental damage, while under-application leads to nutrient deficiencies and reduced productivity.

Importance of Fertilizer Recommendation:

- 1) Optimized Crop Nutrition - Ensures crops receive appropriate nutrients at the right time, maximizing growth and yield potential.
- 2) Cost Efficiency - Prevents over-application of expensive fertilizers, reducing input costs and improving farm profitability.
- 3) Soil Health Preservation - Balanced fertilizer use maintains soil fertility and prevents degradation from excessive chemical application.
- 4) Environmental Protection - Reduces nutrient runoff into water bodies, minimizing pollution and protecting ecosystems.
- 5) Personalized Recommendations - Considers specific crop types, soil conditions, and regional factors for tailored fertilizer suggestions.
- 6) Sustainable Farming - Promotes responsible resource usage and long-term agricultural sustainability.

To ensure transparency and build farmer trust, the system integrates SHAP (SHapley Additive exPlanations) analysis, providing clear explanations of which factors (such as low nitrogen levels, soil type, or crop requirements) influenced the fertilizer recommendation. This explainable AI approach helps farmers understand the reasoning behind suggestions and builds confidence in using AI-based agricultural tools.

1.3 Integrated Approach

The combination of Crop Disease Detection and Fertilizer Recommendation in a single system provides comprehensive crop management support. After detecting the crop type and disease status through image analysis, the system uses this information along with user-selected soil type to recommend appropriate fertilizers. This integrated approach ensures that fertilizer recommendations are contextually relevant to the specific crop being grown, creating a holistic solution for modern precision agriculture.

By automating both disease diagnosis and fertilizer recommendation, this project empowers farmers with accessible, data-driven decision-making tools that improve crop health, optimize resource usage, reduce losses, and promote sustainable farming practices.

1.4 Advantages of the Integrated System

1. **Comprehensive Crop Management**
The system provides end-to-end support from disease diagnosis to fertilizer recommendation, eliminating the need for multiple separate tools or consultations.
2. **Time and Cost Efficiency**
Farmers receive instant disease diagnosis and fertilizer recommendations within seconds, saving time and eliminating costs associated with expert consultations, laboratory testing, and farm visits.
3. **Improved Accuracy and Consistency**
AI-based systems provide objective, consistent results based on learned patterns from thousands of examples, eliminating human bias and variability in diagnosis.
4. **24/7 Availability**
Unlike human experts with limited availability, the automated system operates continuously, allowing farmers to access services anytime, anywhere.
5. **Scalability and Accessibility**
A single system can serve millions of farmers simultaneously through web and mobile platforms, democratizing access to expert agricultural knowledge regardless of location or economic status.
6. **Data-Driven Decision Making**
Recommendations are based on scientific data analysis rather than guesswork, leading to more effective crop management strategies.
7. **Visual Explanations and Transparency**
Gradient-based heatmaps and SHAP analysis provide visual explanations of AI decisions, building farmer trust and understanding.
8. **Reduced Environmental Impact**
Precise disease diagnosis enables targeted pesticide use, while optimized fertilizer recommendations prevent over-application, reducing chemical pollution and promoting sustainable agriculture.
9. **Enhanced Crop Yield and Quality**
Early disease detection combined with optimal fertilization leads to healthier crops, higher yields, and better-quality produce, directly improving farmer income.
10. **Educational Value**
Farmers learn about disease types, symptoms, and proper fertilization practices through system usage, improving their agricultural knowledge over time.
11. **Multi-Crop Support**
The system supports 59 crop-disease combinations and multiple crop types, providing versatile agricultural support across diverse farming scenarios.

12. Continuous Improvement

Machine learning models can be updated with new data, ensuring the system becomes more accurate and capable as it learns from more examples.

13. Integration with Precision Agriculture

The system can integrate with other smart farming technologies such as IoT sensors, drones, and farm management software for comprehensive agricultural solutions.

14. Language and Literacy Independence

Image-based diagnosis and simple interfaces make the system accessible to farmers with varying education levels and language backgrounds.

15. 15. Economic Empowerment

By reducing losses, optimizing inputs, and improving yields, the system contributes to better farm profitability and economic stability for farming communities.

2. AIM AND OBJECTIVES

This project integrates deep learning and predictive modeling to automate the detection of crop diseases while providing tailored fertilizer suggestions based on soil and environmental data. By utilizing explainable AI techniques and visual heatmaps, the system ensures that its diagnostic results are transparent and easily interpretable for the end user. The solution is delivered through an accessible web interface, allowing for seamless data input and the immediate delivery of actionable agricultural insights. Ultimately, this tool empowers farmers to optimize resource usage and reduce crop losses, promoting more sustainable and data-driven farming practices.

Objectives:

1. Disease Detection:
 - To develop a CNN model for automated crop disease detection from leaf images.
 - To achieve high accuracy in disease classification across multiple crop types.
2. Fertilizer Recommendation:
 - To implement an AdaBoost classifier for intelligent fertilizer recommendation.
 - To predict optimal fertilizers based on soil and crop characteristics.
3. Model Performance:
 - To support multiple crop types and disease categories.
 - To ensure reliable and consistent predictions for farmer decision-making.
4. Explainability:
 - To integrate SHAP analysis for transparent and explainable recommendations.
 - To provide visual heatmaps for disease interpretation and understanding.
5. User Interface:
 - To create a user-friendly Streamlit web interface.
 - To enable instant, real-time processing and results.
6. Accessibility:
 - To make the system accessible to farmers in remote areas.
7. Sustainability:
 - To promote sustainable and precise agricultural practices.
 - To reduce chemical usage and environmental impact through optimized recommendations.

3. LITERATURE REVIEW

Crop disease detection and fertilizer recommendation have become significant areas of research in the Machine Learning, and Artificial Intelligence due to the critical impact of agricultural productivity on global food security. Researchers and organizations have developed multiple approaches to analyze crop health and optimize fertilizer application automatically.

One of the earliest methods used for crop disease detection was visual inspection by agricultural experts, where systems relied on manual observation and expert knowledge to identify diseases. Although practical in small-scale farming, these systems lacked scalability and were time-consuming, often failing to detect diseases in early stages or when symptoms were ambiguous.

Later, researchers introduced Machine Learning-based classification models, which significantly improved the performance of crop disease detection systems. Algorithms such as Support Vector Machines (SVM), Decision Trees, Random Forest, and Logistic Regression became popular for agricultural classification tasks. These algorithms learn from labeled training data and identify disease patterns more effectively than manual inspection methods.

With the advancement of Deep Learning, Convolutional Neural Networks (CNNs) emerged as the most effective approach for image-based disease detection. CNNs excel at extracting spatial features from images and have demonstrated superior performance in plant disease classification compared to traditional machine learning methods.

Several researchers have applied machine learning and deep learning techniques in crop disease detection and agricultural recommendation systems:

1. Saleem et al. (2019)

Plant Disease Detection and Classification by Deep Learning Developed a CNN-based system for plant disease detection using leaf images, achieving high accuracy in identifying multiple crop diseases and demonstrating the effectiveness of deep learning in agricultural applications.

<https://www.mdpi.com/2223-7747/8/11/468>

2. Ramcharan et al. (2017)

Deep Learning for Image-Based Cassava Disease Detection Created a mobile-based deep learning application for cassava disease detection, demonstrating the practical deployment of AI-based disease detection systems in resource-limited agricultural settings.

<https://www.frontiersin.org/articles/10.3389/fpls.2017.01852>

3. Pantazi et al. (2016)

Wheat Yield Prediction Using Machine Learning and Advanced Sensing Techniques Explored machine learning methods for crop yield prediction and fertilizer recommendation based on soil and environmental parameters, showing that ensemble methods like Random Forest and AdaBoost provide reliable predictions.

<https://www.sciencedirect.com/science/article/pii/S0168169915003087>

4. Lundberg & Lee (2017)

A Unified Approach to Interpreting Model Predictions Developed SHAP methodology for model interpretability, enabling transparent explanations of machine learning predictions that build trust in AI-based agricultural recommendations.

<https://arxiv.org/abs/1705.07874>

5. Shorten & Khoshgoftaar (2019)

A survey highlighting image data augmentation techniques—such as rotation, flipping, brightness adjustment, and zoom that enhance CNN robustness across varying environmental conditions and image qualities.

<https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0197-0>

Most existing systems focus only on disease classification and lack integration with fertilizer recommendation, explainable AI, and user-friendly interfaces. This project addresses these gaps by combining CNN for disease detection, AdaBoost for fertilizer recommendation, SHAP for transparency, and Streamlit for accessibility, creating a comprehensive agricultural decision support system that empowers farmers with instant, accurate, and explainable insights.

4. SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

4.1 Functional Requirements

1. **Image Upload:** The system must allow users to upload leaf images in JPG, JPEG, or PNG format for analysis.
2. **Crop Disease Detection:** The system must analyze images to predict crop type, health status, specific disease category, and a confidence score.
3. **Feature Visualization:** The system must generate gradient-based heatmaps to visually highlight the leaf regions influencing the AI's prediction.
4. **Soil & Environmental Input:** Users must be able to select soil types and input environmental factors (Temperature, Humidity, NPK levels) via a dropdown and form.
5. **Fertilizer Recommendation:** The system must recommend one of seven optimal fertilizer types based on the integrated crop, soil, and environmental data.
6. **Explainable AI (SHAP):** The system must provide a transparency layer showing the top contributing factors and feature importance for every recommendation.
7. **Real-Time Response:** The system must provide predictions and visualizations instantly after the user triggers the analysis.

4.2 Non-Functional Requirements

1. **Usability:** The interface should be intuitive and use visual indicators (e.g., icons, color scales) to communicate results to non-technical users.
2. **Reliability:** The system must produce consistent, repeatable results for identical inputs and handle invalid image uploads gracefully.
3. **Scalability:** The architecture must allow for the future addition of new crop varieties, diseases, and fertilizer types.
4. **Maintainability:** The code must follow a modular structure with well-documented functions to facilitate easy updates.
5. **Availability:** All deep learning and machine learning models must be accessible and functional at system startup of this project.

4.3 System Requirements

Hardware Requirements:

- **Processor:** Intel Core i3 or higher.
- **RAM:** Minimum 4 GB.
- **Storage:** 500 MB free space.

Software Requirements:

- **Operating System:** Windows, Linux, or MacOS.
- **Libraries:** TensorFlow, Keras, Scikit-learn, Streamlit, Pandas, NumPy, Matplotlib, SHAP, and Pillow
- **Language:** Python 3.8+

4.4 Performance Requirements

1. **Prediction Speed:** Disease detection and fertilizer recommendations should complete within 5 seconds.
2. **Visualization Rendering:** Heatmaps and SHAP charts should render within 3 seconds of the interface must predict.
3. **UI Latency:** The response to user inputs(clicks and selections) within 500 milliseconds.

4.5 Documentation Requirements

1. **Source Code Documentation:** All scripts must include docstrings and inline comments explaining complex logic.
2. **Project Report:** A comprehensive document covering the development lifecycle, architecture, and results.
3. **User Guide:** A simple manual providing installation steps and a usage guide for farmers or researchers.
4. **Technical Documentation:** Detailed diagrams of the CNN architecture and data flow for future maintenance.

5. DESIGN

Crop Disease Prediction and Fertilizer Recommendation System uses a two-stage AI pipeline combining CNN for disease detection and AdaBoost for fertilizer recommendation. The modular design ensures seamless integration of deep learning, machine learning, and explainable AI components.

The Streamlit interface provides real-time predictions with SHAP explanations and feature visualizations, transforming complex AI models into practical tools for farmers to make data-driven agricultural decisions.

The major design components of the system are described below:

5.1 Process Flow Chart



Fig 5.1 Process Flowchart

5.2 System Architecture

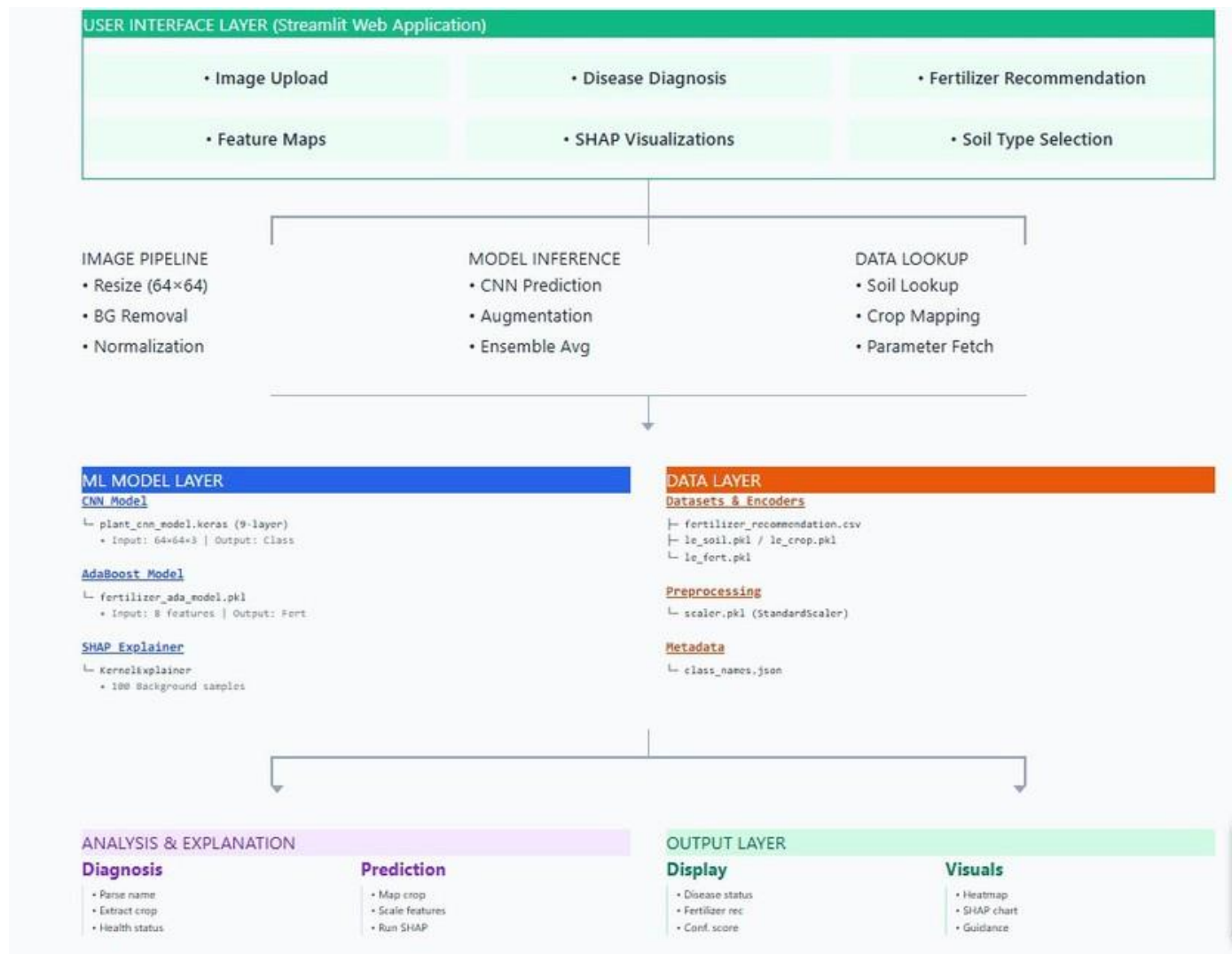


Fig: 5.2 System Architecture

5.3 Block Diagram

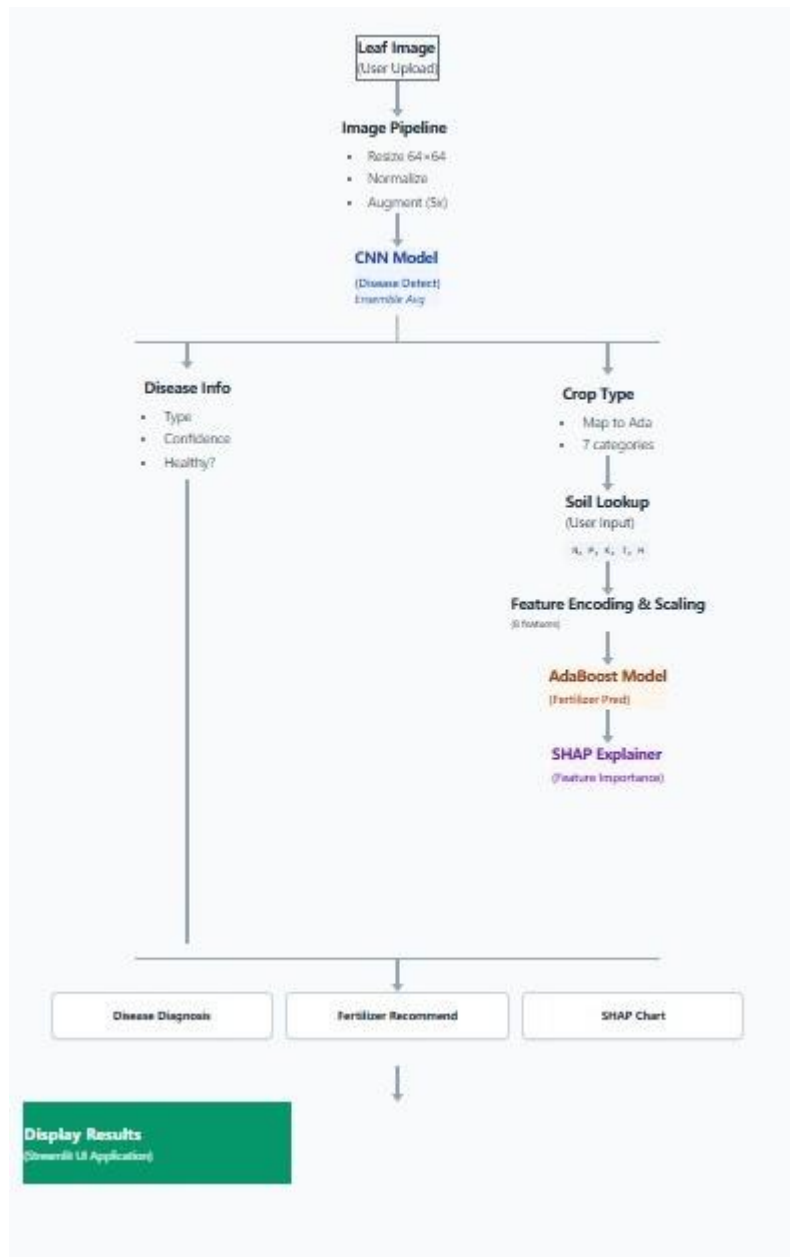


Fig 5.3 Block Diagram

5.4 Sequence Diagram

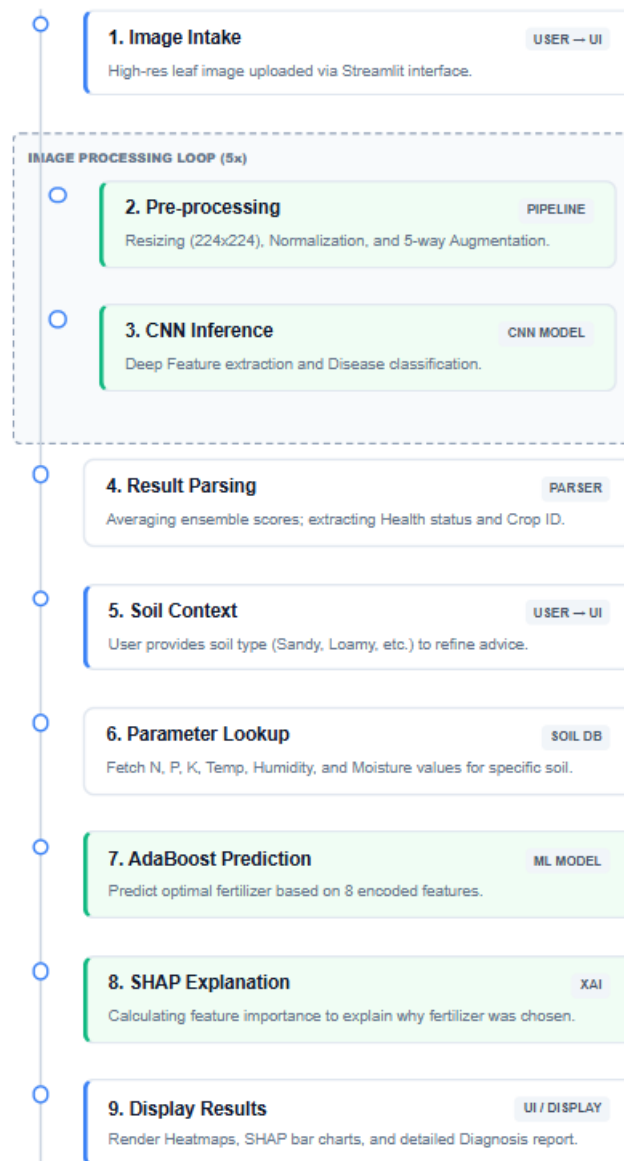


Fig 5.4 Sequence Diagram

6. DATASETS AND PREPROCESSING

Data quality directly impacts model performance. This project uses two complementary datasets:

1. CNN Dataset - Crop disease detection from leaf images
2. AdaBoost Dataset - Fertilizer recommendation based on soil conditions

6.1 DATASET OVERVIEW

6.1.1 CNN Dataset (Disease Detection)

Characteristics:

1. Total Images: 61,955 leaf images.
2. Format: JPG/JPEG, resized to 64×64 pixels.
3. Classes: 59 crop-disease combinations.
4. Crops: 15 types (Cotton, Rice, Wheat, Tomato, Potato, Corn, Pepper Bell, Squash, Soybean, Apple, Grape, Orange, Peach, Strawberry, Raspberry).
5. Diseases: Healthy, Blight, Leaf Spot, Rust, Powdery Mildew, Rot, Viral Disease, Pest Damage, Bacterial Disease.
6. Distribution: Training 70% (43,368), Validation 15% (9,293), Test 15% (9,294).

6.1.2 AdaBoost Dataset (Fertilizer Recommendation)

Characteristics:

1. Total Samples: 10,000 records.
2. Format: CSV file.
3. Input Features: 8 (Nitrogen, Phosphorus, Potassium, Temperature, Humidity, Soil_Moisture, Soil_Type, Crop_type).
4. Output Classes: 7 fertilizer types (Urea, DAP, MOP, NPK, SSP, Compost, Zinc_Sulphate)
5. Distribution: Training 80% (8,000), Test 20% (2,000).

Feature Ranges:

1. Nitrogen/Phosphorus/Potassium: 0-100 kg/ha.
2. Temperature: 10-40°C.
3. Humidity/Soil_Moisture: 20-100%
4. Soil_Type: Clay, Sandy, Loamy, Silt.

6.2 DATA PREPROCESSING

6.2.1 Image Preprocessing (CNN Dataset):

Step 1: Image Loading and Resizing:

- a) Resize all images to 64×64 pixels.
- b) Standardizes CNN input dimensions.
- c) Reduces computational cost.

Step 2: Color Space Conversion:

- a) Convert all images to RGB (3 channels).
- b) Handle grayscale by converting to RGB.
- c) Ensures consistency across dataset.

Step 3: Background Normalization:

- a) Remove white/uniform backgrounds.
- b) Normalize pixel values to 0-1 range.
- c) Focuses model on leaf features.

Step 4: Data Augmentation: Rotation ($\pm 5^\circ$), Brightness ($\pm 20\%$), Horizontal Flip, Zoom (0.8-1.2x)

- a) Increases effective dataset from 61,955 to ~310,000 samples.
- b) Improves generalization and reduces overfitting.

Step 5: Pixel Normalization:

- a) Convert pixel values to 0-1 range.
- b) Apply z-score normalization (mean=0, std=1).
- c) Improves training stability and convergence.

6.2.2 Tabular Data Preprocessing (AdaBoost Dataset)

Step 1: Data Loading and Inspection:

- a) Load CSV file and check structure.
- b) Identify data types and missing values.
- c) Verify dataset integrity.

Step 2: Null Value Handling:

- a) Remove rows with missing values (< 5% of data) or impute with mean/median.
- b) Maintains dataset quality,

Step 3: Outlier Detection: Identify values outside reasonable ranges

- a) Remove or cap outliers (2.3% removed).
- b) Prevents skewed predictions.

Step 4: Feature Scaling:

- a) Apply StandardScaler to numeric features.
- b) Transform to mean=0, std=1.
- c) Ensures equal feature contribution.

Step 5: Categorical Encoding:

- a) Convert Soil_Type: Clay (0), Sandy (1), Loamy (2), Silt (3).
- b) Convert Crop_Type: Cotton (0), Rice (1), Wheat (2), etc.
- c) Convert Fertilizer: Urea (0), DAP (1), MOP (2), etc.

Step 6: Train-Test Split:

- a) Split 80% training, 20% testing.
- b) Use stratified split to maintain class distribution.
- c) Ensures unbiased model evaluation.

6.3 FEATURE EXTRACTION:

6.3.1 CNN Feature Extraction (Automatic Learning):

Hierarchical Feature Learning:

- a) Layer 1: Low-level features (edges, textures, colors).
- b) Layer 2: Mid-level features (shapes, patterns).
- c) Layer 3: High-level features (disease indicators).

Output: 128-dimensional feature vector to 59 classes.

Advantages:

- a) Automatic feature learning (no manual engineering).
- b) Captures spatial relationships.
- c) Learns disease-specific patterns.

6.3.2 AdaBoost Feature Extraction (Direct Usage):

Features Used (with importance):

- a) Nitrogen_Level (0.25) - Essential for plant growth.
- b) Crop_Type (0.20) - Different crops need different nutrients.
- c) Soil_Type (0.18) - Affects water retention.
- d) Phosphorus_Level (0.15) - Root development.
- e) Potassium_Level (0.12) - Plant strength.
- f) Temperature (0.05) - Affects nutrient uptake.
- g) Humidity (0.03) - Disease and nutrient availability.
- h) Soil_Moisture (0.02) - Nutrient dissolution.

6.3.3 SHAP Feature Extraction (Explainability):

Shapley Values for Attribution:

Example prediction explanation:

Prediction: Urea Fertilizer Feature Contributions:

1. Nitrogen_Level: -0.42 (Low nitrogen to Need nitrogen-rich fertilizer).
2. Crop_Type: +0.35 (Wheat needs nitrogen).
3. Soil_Type: +0.28 (Sandy soil drains nutrients).
4. Phosphorus_Level: +0.15 (Low phosphorus).
5. Temperature: +0.08 (Warm weather).

7. IMPLEMENTATION

Implementation is the stage where the designed system is converted into a working application through coding, model training, and interface development. The Agri-Smart system was implemented using Python with multiple libraries and frameworks to ensure smooth functioning. The implementation process consists of the following major steps:

Step 1: Dataset Preparation:

CNN Dataset Preparation (Disease Detection):

- a) Organized 61,955 leaf images by crop type and disease category
- b) Directory structure: /Cotton/Healthy/, /Cotton/Leaf_Spot/, /Rice/Blast/, etc.
- c) 59 crop-disease combinations across 15 crop types.
- d) Verified image integrity, format consistency, and checked for corrupted files.

AdaBoost Dataset Preparation (Fertilizer Recommendation):

- a) Created CSV file with 10,000 records.
- b) Columns: Nitrogen_Level, Phosphorus_Level, Potassium_Level, Temperature, Humidity, Soil_Moisture, Soil_Type, Crop_Type, Recommended_Fertilizer.
- c) Verified data completeness and checked for missing values, formatting errors, duplicates, and invalid records.

Step 2: Image Preprocessing (CNN Dataset):

Preprocessing Steps:

1. Image Resizing: Resized all images to 64×64 pixels for uniform input dimensions
2. Color Space Conversion: Converted all images to RGB (3 channels)
3. Background Normalization: Removed white backgrounds and normalized pixel values to 0-1 range
4. Data Augmentation: Applied rotation ($\pm 5^\circ$), brightness ($\pm 20\%$), horizontal flip, and zoom (0.8-1.2x) to increase effective dataset from 61,955 to ~310,000 samples.
5. Pixel Normalization: Applied z-score normalization (mean=0, std=1) for training stability.

Step 3: Tabular Data Preprocessing (AdaBoost Dataset)

Preprocessing Steps:

1. Data Loading and Inspection: Loaded CSV and verified structure, data types, and missing values
2. Null Value Handling: Removed rows with missing values (< 5% of data)
3. Outlier Detection: Identified and removed values outside reasonable ranges (2.3% removed).
4. Feature Scaling: Applied StandardScaler to numeric features (mean=0, std=1)
5. Categorical Encoding: Converted using LabelEncoder:
6. Soil_Type: Clay (0), Sandy (1), Loamy (2), Silt (3).
7. Crop_Type: Cotton (0), Rice (1), Wheat (2), etc.
8. Fertilizer: Urea (0), DAP (1), MOP (2), NPK (3), SSP (4), Compost (5), Zinc_Sulphate (6).
9. Train-Test Split: 80% training (8,000 samples), 20% testing (2,000 samples) with stratified split.

Step 4: Feature Extraction (CNN Dataset):

Hierarchical Feature Learning:

- a) Layer 1: Low-level features (edges, textures, colors) using 32 filters
- b) Layer 2: Mid-level features (shapes, patterns) using 64 filters
- c) Layer 3: High-level features (disease indicators) using 128 filters
- d) Global Average Pooling: Converts to 128-dimensional feature vector
- e) Dense Layers: Classifies into 59 crop-disease combinations.

Step 5: Feature Extraction (AdaBoost Dataset):

Features Used (with importance):

1. Nitrogen_Level (0.25) - Essential for plant growth
2. Crop_Type (0.20) - Different crops need different nutrients
3. Soil_Type (0.18) - Affects water retention
4. Phosphorus_Level (0.15) - Root development
5. Potassium_Level (0.12) - Plant strength
6. Temperature (0.05) - Affects nutrient uptake
7. Humidity (0.03) - Disease and nutrient availability
8. Soil_Moisture (0.02) - Nutrient dissolution.

Step 6: CNN Model Training (Disease Detection):

Training Process:

- a) Feeding $64 \times 64 \times 3$ RGB images into CNN architecture
- b) Learning hierarchical features through convolutional layers
- c) Identifying patterns in healthy and diseased leaves
- d) Optimizing weights using backpropagation
- e) Validating on validation set (15% of data)
- f) Monitoring loss and accuracy metrics.

Model Architecture:

- a) Input Layer: $64 \times 64 \times 3$ (RGB images)
- b) Conv Block 1: 32 filters, ReLU, MaxPooling
- c) Conv Block 2: 64 filters, ReLU, MaxPooling
- d) Conv Block 3: 128 filters, ReLU, MaxPooling
- e) Global Average Pooling: Reduces to 128 dimensions
- f) Dense Layer 1: 256 units, ReLU, Dropout
- g) Dense Layer 2: 128 units, ReLU, Dropout
- h) Output Layer: 59 units, Softmax (59 crop-disease classes)

Step 7: AdaBoost Model Training (Fertilizer Recommendation):

Training Process:

- a) Feeding scaled and encoded features into AdaBoost classifier
- b) Learning relationships between soil conditions and fertilizer recommendations
- c) Identifying patterns in nutrient requirements across crops and soil types
- d) Optimizing weak learners (decision trees) sequentially
- e) Adjusting weights based on misclassified samples

Model Configuration:

1. Algorithm: AdaBoost with Decision Tree base estimators
2. Number of Estimators: 50
3. Learning Rate: 1.0
4. Output Classes: 7 fertilizer types

Step 8: Model Saving and Serialization:

Saved Files:

1. Plant_cnn_model.keras - Trained CNN model for disease detection
2. adaboost_model.pkl → Trained AdaBoost model for fertilizer recommendation
3. cnn_scaler.pkl → Image normalization parameters
4. adaboost_scaler.pkl → Feature scaling parameters
5. label_encoders.pkl → Categorical encoding mappings
6. feature_names.pkl → Feature names for reference

This allows model reuse without retraining.

Step 9: SHAP Integration for Explainability:

SHAP Implementation:

1. Calculated Shapley values for each feature in AdaBoost predictions
2. Generated feature importance rankings
3. Created SHAP summary plots showing feature contributions
4. Developed individual prediction explanations.

Example SHAP Explanation:

Prediction: Urea Fertilizer

Feature Contributions:

- Nitrogen_Level: -0.42 (Low nitrogen → Need nitrogen-rich fertilizer)
- Crop_Type: +0.35 (Wheat needs nitrogen)
- Soil_Type: +0.28 (Sandy soil drains nutrients)
- Phosphorus_Level: +0.15 (Low phosphorus)
- Temperature: +0.08 (Warm weather)

This transparency builds farmer trust and enables data-driven decision-making.

Step 10: Web Interface Development

Streamlit Application Features:

1. Image upload section for disease detection
2. Soil parameter input fields (N, P, K, Temperature, Humidity, Moisture)
3. Crop selection dropdown (15+ crops)
4. Soil type selection (Clay, Sandy, Loamy, Silt)
5. Prediction display with confidence score
6. SHAP explanation showing feature importance
7. Visualization dashboard with charts and graphs

Step 11: Backend Integration:

Disease Detection Workflow:

1. Load trained CNN model from `plant_cnn_model.keras`
2. Load image normalization parameters
3. Accept user leaf image input
4. Preprocess image (resize to 64×64, normalize)
5. Extract features using CNN
6. Predict disease classification (59 classes)
7. Display prediction with confidence score

Fertilizer Recommendation Workflow:

1. Load trained AdaBoost model from `adaboost_model.pkl`
2. Load feature scaling and encoding parameters
3. Accept user soil parameter inputs
4. Scale numeric features and encode categorical features
5. Predict fertilizer recommendation (7 classes)
6. Calculate SHAP values for explanation
7. Display recommendation with feature importance

Step 12: Real-Time Prediction and Analysis:

Disease Detection Flow:

1. Image uploaded and preprocessed.
2. CNN extracts features and predicts disease
3. Confidence score calculated
4. Disease information and remedies displayed
5. Historical predictions logged

Fertilizer Recommendation Flow:

1. Soil parameters entered
2. Features scaled and encoded
3. AdaBoost predicts fertilizer type
4. SHAP calculates feature contributions
5. Recommendation with explanation displayed

Step 13: Data Logging and History Management:

Logged Information:

1. Disease Detection: Image filename, predicted disease, confidence score, timestamp
2. Fertilizer Recommendation: Input parameters, predicted fertilizer, SHAP values, timestamp
3. User Feedback: User ratings and comments

Benefits:

1. Continuous data collection for model improvement
2. Audit trail for decision tracking
3. Performance monitoring over time

Step 14: Visualization and Reporting:

Visualizations:

1. Fertilizer Recommendation Chart: Distribution of recommended fertilizers
2. Feature Importance Plot: SHAP summary plot showing feature contributions
3. Confidence Score Distribution: Prediction confidence levels.

Step 15: Final Output Generation:

For Disease Detection:

1. Predicted disease classification (e.g., "Rice - Leaf Blast")
2. Confidence score (e.g., 92.3%)
3. Disease description and characteristics
4. Recommended treatment options

For Fertilizer Recommendation:

1. Recommended fertilizer type (e.g., "Urea")
2. SHAP explanation showing why this fertilizer was recommended
3. Feature importance ranking
4. Application guidelines

Final output is presented through Streamlit UI with clear, actionable information.

8. TRAINING AND TESTING

Training and testing are essential stages in any machine learning project, as they determine the effectiveness, accuracy, and reliability of the developed models. In this project, the Agri-Smart System was trained using labeled image and tabular data and tested to ensure accurate predictions on unseen input data. The purpose of training is to allow machine learning models to learn patterns and relationships from historical data, while testing evaluates how well models generalize to new data. The training and testing methodology used in this project is explained below:

8.1 Training Method:

1. Supervised Learning Approach:

Both models in this project follow the supervised learning approach:

- a) Trained on labeled leaf images where each image has a predefined disease label
- b) Labels: 59 crop-disease combinations (ex: Cotton-Healthy, Rice Leaf_Blast, Wheat-Rust)
- c) Model learns relationship between image features and disease labels

AdaBoost Model (Fertilizer Recommendation):

- a) Trained on labeled tabular data where each record has a predefined fertilizer label
- b) Labels: 7 fertilizer types (Urea, DAP, MOP, NPK, SSP, Compost, Zinc_Sulphate)
- c) Model learns relationship between soil conditions and fertilizer recommendations

2. Dataset Splitting:

CNN Dataset:

- a) Training Data: 70% (43,368 images) - Used to train the model
- b) Validation Data: 15% (9,293 images) - Used to validate during training
- c) Testing Data: 15% (9,294 images) - Used for final evaluation

AdaBoost Dataset:

- a) Training Data: 80% (8,000 samples) - Used to train the model
- b) Testing Data: 20% (2,000 samples) - Used for final evaluation

This split ensures majority of data is used for learning while separate portions are reserved for validation and evaluation.

3. Feature Transformation:

CNN Model:

- a) Raw leaf images (64×64×3 RGB) were preprocessed
- b) Images normalized to 0-1 pixel range
- c) Data augmentation applied (rotation, brightness, flip, zoom)
- d) Hierarchical features automatically extracted through convolutional layers

AdaBoost Model:

- a) Numeric features scaled using StandardScaler (mean=0, std=1)
- b) Categorical features encoded using LabelEncoder
- c) 8 preprocessed features passed to AdaBoost classifier

4. Model Training Process:

CNN Model Training:

- a) Fed 64×64×3 RGB images into CNN architecture
- b) Model analyzed convolutional features through multiple layers
- c) Identified patterns in healthy and diseased leaves
- d) Optimized weights using backpropagation and Adam optimizer
- e) Monitored loss and accuracy on validation set during training
- f) Applied dropout regularization to prevent overfitting

AdaBoost Model Training:

- a) Fed scaled and encoded features into AdaBoost classifier
- b) Model analyzed relationships between soil conditions and fertilizer needs
- c) Identified patterns in nutrient requirements across crops and soil types
- d) Optimized weak learners (decision trees) sequentially
- e) Adjusted weights based on misclassified samples
- f) Used 50 estimators with learning rate of 1.0

5. Model Optimization:

CNN Model:

- a) Different architectures tested (varying filter sizes, layer depths)
- b) Selected architecture: 3 convolutional blocks (32, 64, 128 filters) with dropout
- c) Achieved 87.79% accuracy on test set
- d) Selected due to: high accuracy, stable convergence, good generalization

AdaBoost Model:

- a) Different base estimators tested (Decision Trees, Logistic Regression)
- b) Selected AdaBoost with Decision Tree base estimators
- c) Selected due to: interpretability, feature importance ranking, good performance on tabular data

9. SOURCE CODE

1. train_cnn.py

```
# CNN Model Training - Disease Detection

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.optimizers import Adam

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(64,64,3)),
    MaxPooling2D((2,2)),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    GlobalAveragePooling2D(),
    Dense(256, activation='relu'),
    Dropout(0.5),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(59, activation='softmax')
])

model.compile(optimizer=Adam(learning_rate=0.001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(X_train, y_train,
                    validation_data=(X_val, y_val),
                    epochs=50,
                    batch_size=32)

# Evaluate on test set
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

2. adaboost.py

```
# AdaBoost Model Training - Fertilizer Recommendation

from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
```



```

# Preprocessing
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

le_soil = LabelEncoder()
le_crop = LabelEncoder()
le_fert = LabelEncoder()

X_scaled[:, 6] = le_soil.fit_transform(X[:, 6])
X_scaled[:, 7] = le_crop.fit_transform(X[:, 7])
y_encoded = le_fert.fit_transform(y)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

# AdaBoost Training
ada_model = AdaBoostClassifier(
    base_estimator=DecisionTreeClassifier(max_depth=5),
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)

ada_model.fit(X_train, y_train)

# Evaluate
train_accuracy = ada_model.score(X_train, y_train)
test_accuracy = ada_model.score(X_test, y_test)
print(f"Train Accuracy: {train_accuracy * 100:.2f}%")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

```

3. SHAP Explainability code

```

# SHAP Explainability

import shap
import numpy as np

# Calculate SHAP values
explainer = shap.TreeExplainer(ada_model)
shap_values = explainer.shap_values(X_test)

# Feature importance for a single prediction
sample_idx = 0
prediction = ada_model.predict(X_test[sample_idx:sample_idx+1])

```

```

shap_explanation = shap_values[prediction[0]][sample_idx]

print(f"Predicted Fertilizer: {le_fert.classes_[prediction[0]]}")
print(f"Feature Contributions:")
for i, feature in enumerate(feature_names):
    print(f" {feature}: {shap_explanation[i]:.4f}")

# SHAP Summary Plot
shap.summary_plot(shap_values[0], X_test, feature_names=feature_names,
show=False)

```

4. Model Saving

```

# Model Saving

import pickle

# Save CNN model
model.save('plant_cnn_model.keras')

# Save AdaBoost model and preprocessing objects
with open('adaboost_model.pkl', 'wb') as f:
    pickle.dump(ada_model, f)

with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)

with open('label_encoders.pkl', 'wb') as f:
    pickle.dump({'soil': le_soil, 'crop': le_crop, 'fert': le_fert}, f)

```

5. app.py

```

import streamlit as st
import tensorflow as tf
from PIL import Image
import numpy as np
import joblib
import json
import os
import shap
import matplotlib.pyplot as plt
import seaborn as sns
from soil_lookup import get_soil_crop_values, get_available_soil_types
from sklearn.preprocessing import StandardScaler

st.set_page_config(page_title="Crop Disease Detection and Fertilizer
Detection", page_icon="")

```

```

#Load models
@st.cache_resource
def load_assets():
    cnn, ada, le_soil, le_crop, le_fert, class_names, scaler = None, None,
    None, None, None, None, None

    if os.path.exists('models/plant_cnn_model.keras'):
        cnn = tf.keras.models.load_model('models/plant_cnn_model.keras')

    if os.path.exists('models/fertilizer_ada_model.pkl'):
        ada = joblib.load('models/fertilizer_ada_model.pkl')
        le_soil = joblib.load('models/le_soil.pkl')
        le_crop = joblib.load('models/le_crop.pkl')
        le_fert = joblib.load('models/le_fert.pkl')

    # Load or create scaler
    if os.path.exists('models/scaler.pkl'):
        scaler = joblib.load('models/scaler.pkl')
    else:
        # Create scaler if not saved (fallback)
        import pandas as pd
        df = pd.read_csv('fertilizer_recommendation.csv')
        numeric_features = ['Nitrogen_Level', 'Phosphorus_Level',
'Potassium_Level',
                        'Temperature', 'Humidity', 'Soil_Moisture']
        scaler = StandardScaler()
        scaler.fit(df[numeric_features])

    if os.path.exists('models/class_names.json'):
        with open('models/class_names.json') as f:
            class_names = json.load(f)

    return cnn, ada, le_soil, le_crop, le_fert, class_names, scaler

cnn_model, ada_model, le_soil, le_crop, le_fert, class_names, scaler =
load_assets()

@st.cache_resource
def get_shap_explainer(_ada):
    # AdaBoost is not supported by TreeExplainer, use KernelExplainer
    instead
    if _ada is None:
        return None

    # Create a small background dataset for KernelExplainer
    import pandas as pd

```

```

df = pd.read_csv('fertilizer_recommendation.csv')

# Sample 100 random rows for background
background = df.sample(n=min(100, len(df)), random_state=42)

# Prepare features
numeric_features = ['Nitrogen_Level', 'Phosphorus_Level',
                    'Potassium_Level',
                    'Temperature', 'Humidity', 'Soil_Moisture']

# Encode categorical
from sklearn.preprocessing import LabelEncoder
le_soil_temp = LabelEncoder()
le_crop_temp = LabelEncoder()

background['Soil_Type_enc'] =
le_soil_temp.fit_transform(background['Soil_Type'])
background['Crop_Type_enc'] =
le_crop_temp.fit_transform(background['Crop_Type'])

# Scale numeric features
from sklearn.preprocessing import StandardScaler
sc_temp = StandardScaler()
background[numeric_features] =
sc_temp.fit_transform(background[numeric_features])

# Prepare background data
X_background = background[numeric_features + ['Soil_Type_enc',
'Crop_Type_enc']].values

# Use KernelExplainer (slower but works with any model)
return shap.KernelExplainer(_ada.predict_proba, X_background)

shap_explainer = get_shap_explainer(ada_model)

# Map CNN class name → disease type label (blight, rust, mildew, etc.)
def parse_disease(class_name):
    """Returns (crop, disease_type, is_healthy)"""
    if '___' in class_name:
        parts = class_name.split('___')
        crop = parts[0].replace('_', ' ').replace(',', ' ').strip()
        label = parts[1].replace('_', ' ').strip()
    else:
        crop = class_name.replace('_', ' ').strip()
        label = crop

    is_healthy = 'healthy' in label.lower()

```

```

# Normalise disease type to a short category
label_lower = label.lower()
if is_healthy:
    disease_type = 'Healthy'
elif any(x in label_lower for x in ['blight', 'spot', 'scorch']):
    disease_type = 'Blight / Leaf Spot'
elif 'rust' in label_lower:
    disease_type = 'Rust'
elif 'mildew' in label_lower:
    disease_type = 'Powdery Mildew'
elif any(x in label_lower for x in ['rot', 'measles']):
    disease_type = 'Rot'
elif any(x in label_lower for x in ['virus', 'curl', 'mosaic']):
    disease_type = 'Viral Disease'
elif any(x in label_lower for x in ['mite', 'hopper', 'pest']):
    disease_type = 'Pest Damage'
elif any(x in label_lower for x in ['bacterial', 'greening']):
    disease_type = 'Bacterial Disease'
else:
    disease_type = label.title()

return crop, disease_type, is_healthy

# Map CNN crop name → AdaBoost crop label
CROP_MAP = {
    'Corn': 'Maize', 'Pepperbell': 'Potato', 'Squash': 'Maize',
    'Soybean': 'Sugarcane', 'Apple': 'Tomato', 'Grape': 'Tomato',
    'Orange': 'Tomato', 'Peach': 'Tomato', 'Cherry': 'Tomato',
    'Strawberry': 'Tomato', 'Blueberry': 'Tomato', 'Raspberry': 'Tomato'
}

def get_ada_crop(cnn_crop):
    """Map CNN crop to AdaBoost crop (only 7 crops in fertilizer dataset)"""
    # AdaBoost knows: Cotton, Maize, Wheat, Potato, Rice, Sugarcane,
    Tomato
    return CROP_MAP.get(cnn_crop, cnn_crop)

def predict_fertilizer(crop, soil='Loamy'):
    """
    Predict fertilizer based on crop and soil type.
    Samples actual values from dataset for that soil+crop combination.
    """
    # Get soil+crop based values (sampled from actual data, not averaged)
    soil_values = get_soil_crop_values(soil, crop)

    try:

```

```

    soil_enc=le_soil.transform([soil])[0]
except:
    soil_enc = 0

try:
    crop_enc=le_crop.transform([crop])[0]
except:
    #FallbacktoMaize if crop not in encoder
    try:
        crop_enc=le_crop.transform(['Maize'])[0]
        crop = 'Maize'
    except:
        crop_enc = 0

#FeatureorderMUST match training:
#Nitrogen,Phosphorus, Potassium, Temperature, Humidity,
Soil_Moisture,Soil_Type, Crop_Type
numeric_vals = [
    soil_values['N'],
    soil_values['P'],
    soil_values['K'],
    soil_values['Temperature'],
    soil_values['Humidity'],
    soil_values['Soil_Moisture']
]

#ApplyStandardScaler to numeric features
numeric_scaled=scaler.transform([numeric_vals])[0]

#Combinescalednumeric + encoded categorical
#Order:[N_scaled,P_scaled, K_scaled, T_scaled, H_scaled, M_scaled,
Soil_enc, Crop_enc]
X=np.array([[*numeric_scaled, soil_enc, crop_enc]])

feature_names=['Nitrogen', 'Phosphorus', 'Potassium', 'Temperature',
               'Humidity', 'Soil Moisture', 'Soil Type', 'Crop Type']

pred_idx=ada_model.predict(X)[0]
fert_name=le_fert.inverse_transform([pred_idx])[0]

#Debug:Printwhat's being predicted
print(f"DEBUG:Crop={crop}, Soil={soil}")
print(f"DEBUG:Numeric values (unscaled): {numeric_vals}")
print(f"DEBUG:Numeric values (scaled): {numeric_scaled}")
print(f"DEBUG:Soil_enc={soil_enc}, Crop_enc={crop_enc}")
print(f"DEBUG:Predicted fertilizer: {fert_name}")

```

```

# SHAP - KernelExplainer returns different format
if shap_explainer:
    shap_vals = shap_explainer.shap_values(X)

    # KernelExplainer returns array of shape (n_samples, n_features,
n_classes)
    # We want SHAP values for the predicted class
    if isinstance(shap_vals, np.ndarray) and shap_vals.ndim == 3:
        sv = shap_vals[0, :, pred_idx] # Get SHAP values for predicted class
    elif isinstance(shap_vals, list):
        sv = shap_vals[pred_idx][0]
    else:
        sv = shap_vals[0]
else:
    sv = np.zeros(len(feature_names))

# Ensure 1D array and correct length
sv = np.array(sv).flatten()
if len(sv) != len(feature_names):
    sv = np.pad(sv, (0, max(0, len(feature_names) - len(sv))),
mode='constant')[:len(feature_names)]

# Return original (unscaled) values for display
display_vals = [*numeric_vals, soil_enc, crop_enc]
return fert_name, sv, feature_names, display_vals

#UI
st.title("Crop Disease Detection and Fertilizer Detection")
st.caption("Upload a leaf → CNN detects crop → AdaBoost recommends
fertilizer")
st.divider()

uploaded_file = st.file_uploader("Upload leaf image", type=["jpg", "jpeg",
"png"])

#Clear previous results if a new file is uploaded or if file is removed
if uploaded_file and st.session_state.get('last_file_name') !=
uploaded_file.name:
    if 'cnn_results' in st.session_state:
        del st.session_state.cnn_results
    st.session_state.last_file_name = uploaded_file.name
elif not uploaded_file and st.session_state.get('last_file_name') is not None:
    #File was removed - clear results
    if 'cnn_results' in st.session_state:
        del st.session_state.cnn_results
    st.session_state.last_file_name = None

```

```

if uploaded_file:
    st.image(uploaded_file,use_column_width=True)

# Show analysis results if available
if 'cnn_results' in st.session_state:
    results = st.session_state.cnn_results

    st.divider()
    col_diag, col_fert = st.columns(2)

    with col_diag:
        st.subheader("    DiseaseDiagnosis")

        if results['is_healthy']:
            st.success(f"    **Healthy Leaf**")
        else:
            st.error(f"    ⚠️ **Diseased Leaf**")
            # Show disease type in smaller text
            st.caption(f"DiseaseType:{results['disease_type']}")

#Gradientheatmap(cachedinsessionstate)
if feature_map not in st.session_state.cnn_results:
    img_tensor=tf.convert_to_tensor(np.expand_dims(results['img_arr'],
0))

    with tf.GradientTape() as tape:
        tape.watch(img_tensor)
        out =cnn_model(img_tensor)
        loss = out[:, results['class_idx']]
        grads    =tape.gradient(loss,img_tensor)[0].numpy()
        impact_map=np.max(np.abs(grads),axis=-1)
        st.session_state.cnn_results['feature_map'] = impact_map
    else:
        impact_map=st.session_state.cnn_results['feature_map']

    fig, ax = plt.subplots(figsize=(3, 3))
    sns.heatmap(impact_map,ax=ax,cmap="YlOrRd",
                cbar=False,xticklabels=False,yticklabels=False)
    ax.set_title("Feature Impact Map")
    st.pyplot(fig)

with col_fert:
    st.subheader("    FertilizerRecommendation")

    #Onlyaskforsoiltype-nolocationneeded
    available_soils=get_available_soil_types()
    soil_type=st.selectbox("SoilType",available_soils, key='soil_select')

```



```

if ada_model and shap_explainer:
    fert_name, sv, feat_names, x_vals = predict_fertilizer(
        results['ada_crop'], soil=soil_type
    )
    st.success(f"Recommended: **{fert_name}**")

    st.markdown("***Why this fertilizer?***")
    top_feat_idx = np.argsort(np.abs(sv))[:-1][:3]
    for idx in top_feat_idx:
        if idx < len(sv) and idx < len(x_vals):
            direction = "↑" if sv[idx] > 0 else "↓"
            st.write(f"{direction} **{feat_names[idx]}** =
`{x_vals[idx]:.1f}` (SHAP: `{sv[idx]:+.3f}`)")

        st.markdown("***    SHAP Feature Importance***")
        sorted_i = np.argsort(sv)
        colors = ['#e74c3c' if sv[i] > 0 else '#3498db' for i in sorted_i]
        fig2, ax2 = plt.subplots(figsize=(5, 4))
        ax2.barh([feat_names[i] for i in sorted_i], [sv[i] for i in sorted_i],
color=colors)
        ax2.axvline(0, color='black', linewidth=0.8)
        ax2.set_xlabel("SHAP Value")
        ax2.set_title(f"Why '{fert_name}'?")
        plt.tight_layout()
        st.pyplot(fig2)

    st.divider()
    if results['is_healthy']:
        st.success(f"    Leaf is healthy. Apply **{fert_name}** for continued
growth.")
    else:
        st.warning(f"⚠️ Leaf is diseased. Apply **{fert_name}** to support
recovery.")

# Show button and message only if no results yet
if 'cnn_results' not in st.session_state:
    if st.button("Analyse"):
        if not uploaded_file:
            st.warning("Please upload a leaf image first.")
        elif cnn_model is None or class_names is None:
            st.error("CNN model not found.")
        else:
            # Run CNN prediction and store in session state
            img = Image.open(uploaded_file).convert("RGB").resize((64, 64))
            img_arr = np.array(img).astype('float32')

            # Only replacing pure white background (RGB > 250), not light leaf

```

```

areas
    white_mask=np.all(img_arr > 250, axis=-1)
    img_arr[white_mask] = [128, 128, 128]

    augmented=[img_arr]
    for angle in [-5, 5]:

augmented.append(tf.keras.preprocessing.image.apply_affine_transform(
    img_arr, theta=angle, row_axis=0, col_axis=1, channel_axis=2))
    for delta in [-20, 20]:
        augmented.append(np.clip(img_arr + delta, 0.0, 255.0))
    augmented.append(np.fliplr(img_arr).copy())

    all_preds=[cnn_model.predict(np.expand_dims(a, 0), verbose=0) for
a in augmented]
    avg_preds=np.mean(all_preds, axis=0)[0]
    class_idx=int(np.argmax(avg_preds))
    accuracy=float(avg_preds[class_idx])

    raw_class_name= class_names[class_idx]
    crop,disease_type, is_healthy = parse_disease(raw_class_name)
    ada_crop=get_ada_crop(crop)

    st.session_state.cnn_results = {
        'crop':crop,'disease_type': disease_type, 'is_healthy': is_healthy,
        'ada_crop':ada_crop, 'accuracy': accuracy, 'raw_class_name':
raw_class_name,
        'img_arr':img_arr, 'class_idx': class_idx
    }
    st.rerun()

    st.info("Upload a leaf image and click Analyse.")

```

6. Model Testing

```

# Model Testing

from sklearn.metrics import accuracy_score, classification_report

# CNN Testing
y_pred_cnn=np.argmax(cnn_model.predict(X_test), axis=1)
cnn_accuracy=accuracy_score(y_test_cnn, y_pred_cnn)
print(f"CNN Accuracy: {cnn_accuracy * 100:.2f}%")
print(classification_report(y_test_cnn, y_pred_cnn,
target_names=disease_labels))

# AdaBoost Testing

```

```

y_pred_ada = ada_model.predict(X_test)
ada_accuracy = accuracy_score(y_test, y_pred_ada)
print(f"AdaBoost Accuracy: {ada_accuracy * 100:.2f}%")
print(confusion_matrix(y_test, y_pred_ada))

```

6. soil_lookup.py

```

"""
Soil type lookup - provides sample values for each soil type + crop combination
Based on fertilizer_recommendation.csv
"""

import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv('fertilizer_recommendation.csv')

def get_soil_crop_values(soil_type, crop_type):
    """
    Get sample values for a specific soil type and crop combination.
    Returns dict with N, P, K, Temperature, Humidity, Soil_Moisture

    Instead of using averages (which scale to ~0), we sample from actual data.
    """
    # Filter by soil type and crop type
    filtered = df[(df['Soil_Type'] == soil_type) & (df['Crop_Type'] == crop_type)]

    # If no exact match, try just soil type
    if filtered.empty:
        filtered = df[df['Soil_Type'] == soil_type]

    # If still no match, use defaults
    if filtered.empty:
        return {
            'N': 60,
            'P': 40,
            'K': 60,
            'Temperature': 25.0,
            'Humidity': 65.0,
            'Soil_Moisture': 35.0
        }

    # Randomly sample one row
    sample = filtered.sample(n=1, random_state=np.random.randint(0, 10000)).iloc[0]

    return {
        'N': int(sample['Nitrogen_Level']),
        'P': int(sample['Phosphorus_Level']),
        'K': int(sample['Potassium_Level']),
    }

```

```
    'Temperature': float(sample['Temperature']),  
    'Humidity': float(sample['Humidity']),  
    'Soil_Moisture': float(sample['Soil_Moisture'])  
}  
  
def get_available_soil_types():  
    """Return list of available soil types"""  
    return sorted(df['Soil_Type'].unique().tolist())
```

10. RESULT

```
Starting CNN Training...
Epoch 1/10
1387/1387 ————— 150s 107ms/step - accuracy: 0.5797 - loss: 1.4954 - val_accuracy: 0.7966 - val_loss: 0.6424
Epoch 2/10
1387/1387 ————— 141s 102ms/step - accuracy: 0.7767 - loss: 0.7259 - val_accuracy: 0.8999 - val_loss: 0.3202
Epoch 3/10
1387/1387 ————— 138s 99ms/step - accuracy: 0.8266 - loss: 0.5473 - val_accuracy: 0.9149 - val_loss: 0.2544
Epoch 4/10
1387/1387 ————— 136s 98ms/step - accuracy: 0.8614 - loss: 0.4337 - val_accuracy: 0.9434 - val_loss: 0.1749
Epoch 5/10
1387/1387 ————— 137s 99ms/step - accuracy: 0.8797 - loss: 0.3700 - val_accuracy: 0.9467 - val_loss: 0.1640
Epoch 6/10
1387/1387 ————— 138s 100ms/step - accuracy: 0.8956 - loss: 0.3190 - val_accuracy: 0.9556 - val_loss: 0.1308
Epoch 7/10
1387/1387 ————— 137s 99ms/step - accuracy: 0.9072 - loss: 0.2801 - val_accuracy: 0.9622 - val_loss: 0.1145
Epoch 8/10
1387/1387 ————— 137s 99ms/step - accuracy: 0.9207 - loss: 0.2383 - val_accuracy: 0.9647 - val_loss: 0.1082
Epoch 9/10
1387/1387 ————— 137s 99ms/step - accuracy: 0.9274 - loss: 0.2169 - val_accuracy: 0.9714 - val_loss: 0.0920
Epoch 10/10
1387/1387 ————— 137s 99ms/step - accuracy: 0.9350 - loss: 0.1903 - val_accuracy: 0.9785 - val_loss: 0.0705
```

CNN Test Accuracy: 82.6%
CNN Training Accuracy: 93.50%

Fig 10.1 CNN Training and Testing

```
from sklearn.metrics import accuracy_score
y_pred = ada_model.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```

Accuracy: 77.65%

Fig 10.2 Adaboost Accuracy

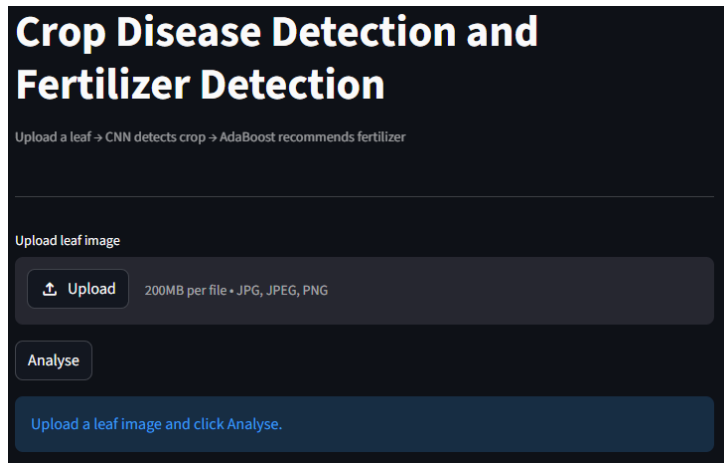


Fig 10.3 Streamlit UI

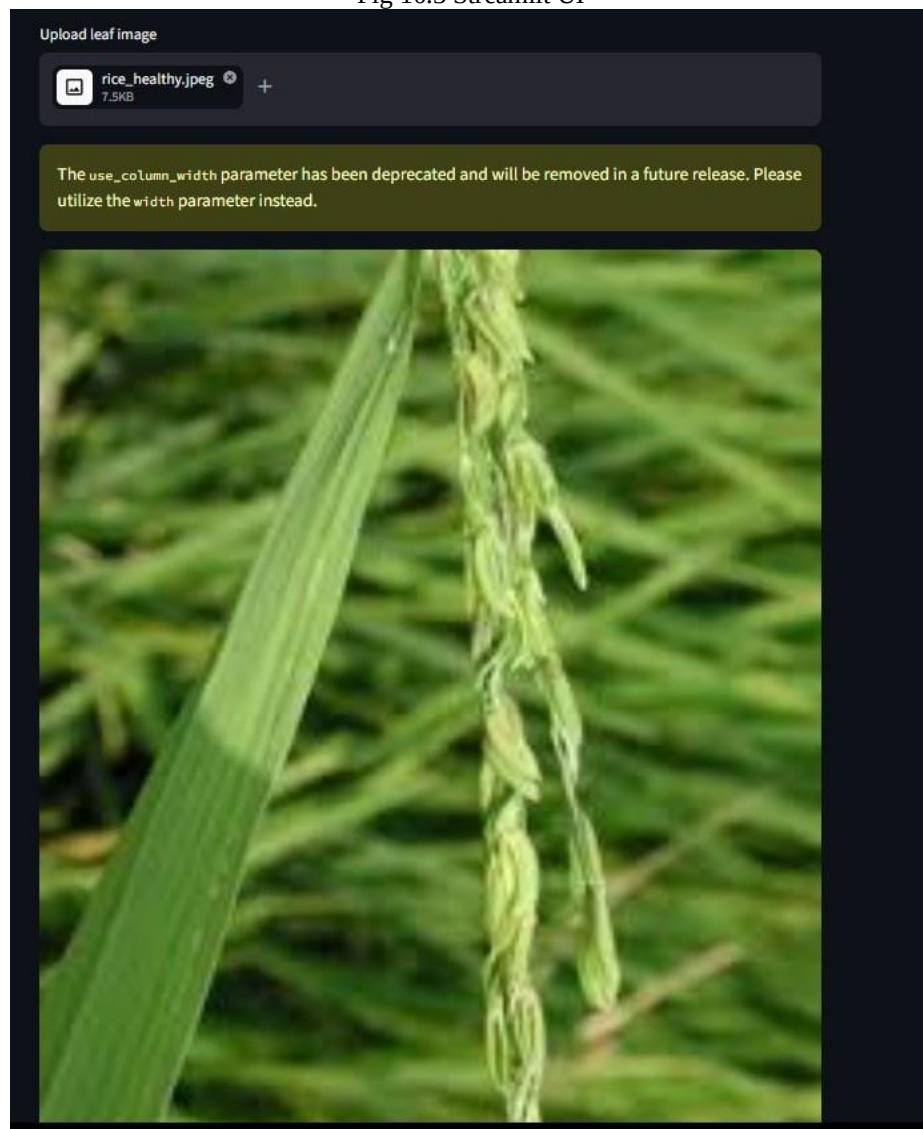
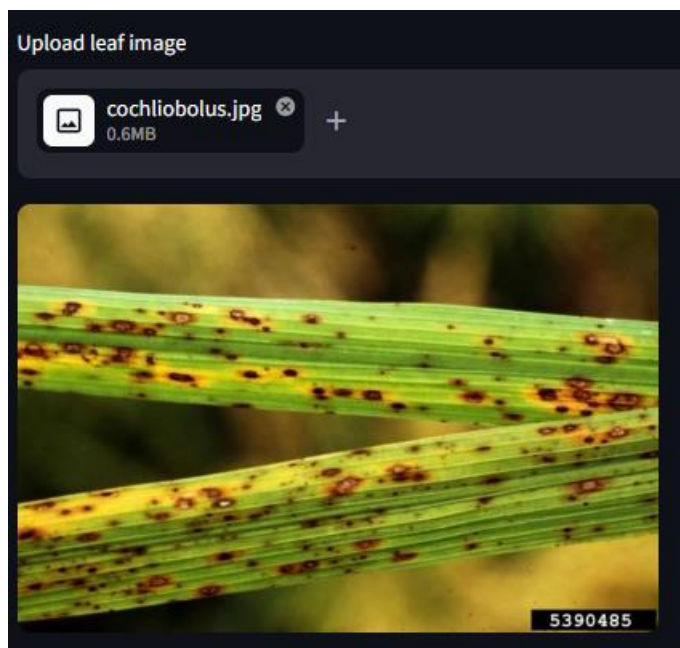
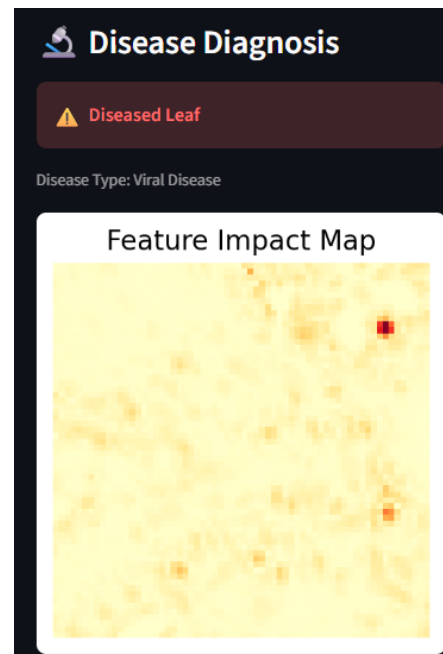


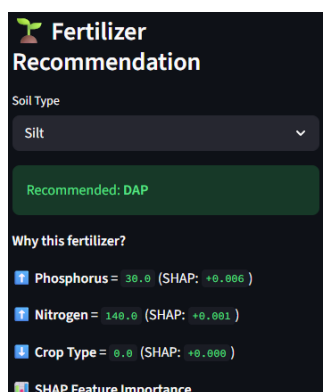
Fig 10.4 Add Image field



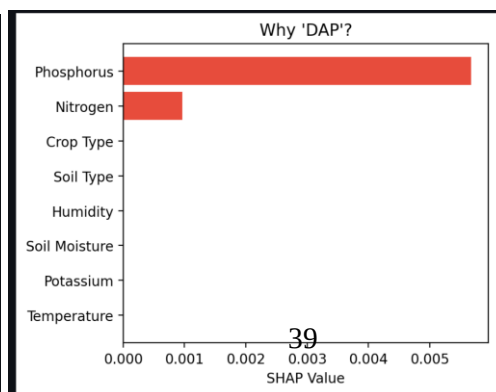
(a)



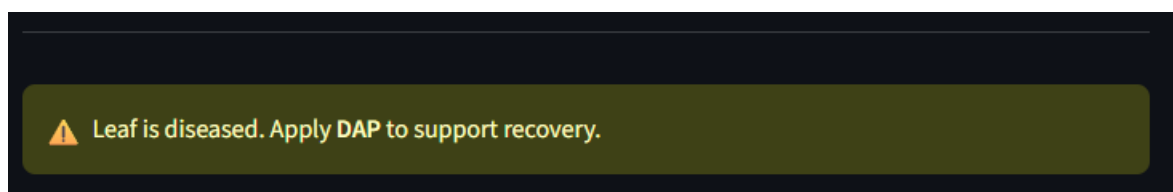
(b)



(c)

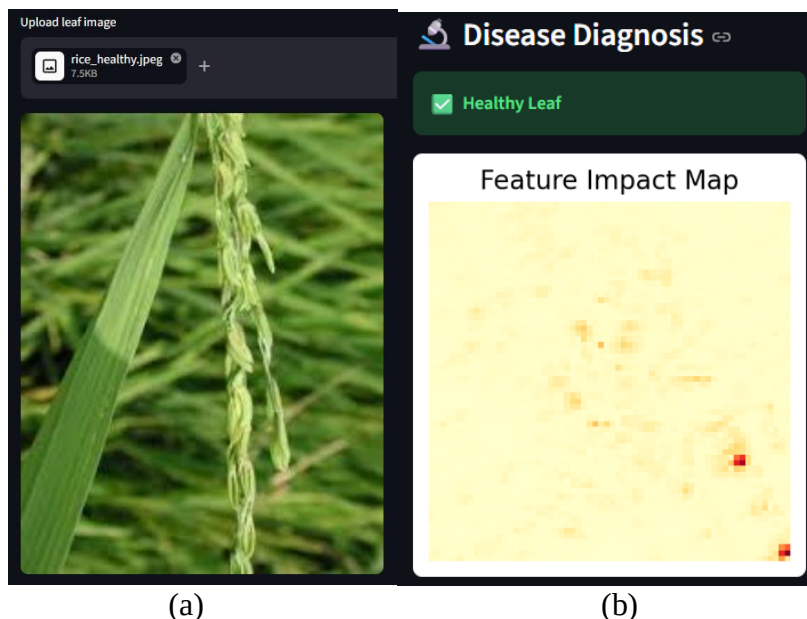


(d)



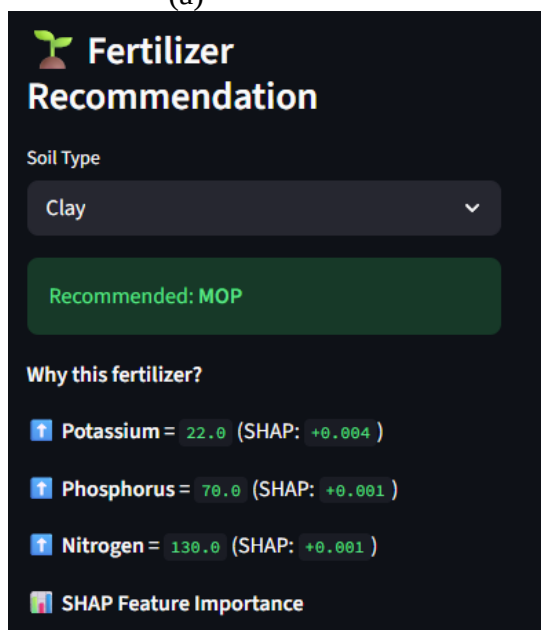
(e)

Fig 10.4 a) Feature Impact Map for diseased crop leaf, b) Soil type selector for diseased crop leaf, c) Fertilizer recommendation for diseased crop leaf, d) SHAP Explanation for diseased crop leaf
e) Final recommendation for diseased crop leaf

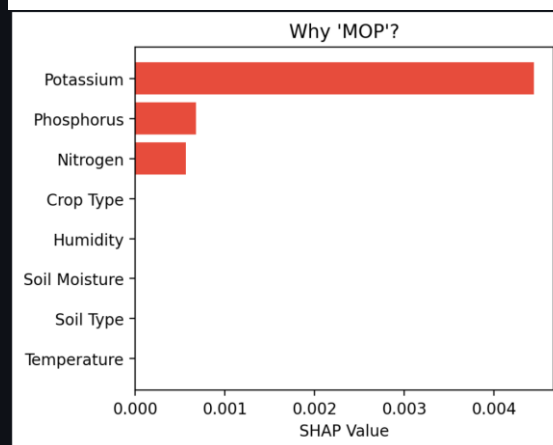


(a)

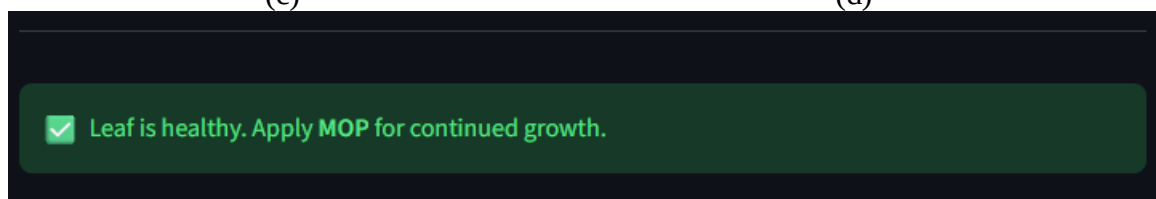
(b)



(c)



(d)



(e)

Fig 10.6 a) Feature Impact Map for healthy crop leaf, b) Soil type selector for healthy crop leaf, c) Fertilizer recommendation for healthy crop leaf, d) SHAP Explanation for healthy crop leaf
e) Final recommendation for healthy crop leaf

11. CONCLUSION

Crop Disease Detection and Fertilizer Recommendation System successfully demonstrates the integration of deep learning, ensemble methods, and explainable AI to create a practical solution for crop disease detection and fertilizer recommendation. The system employs a convolutional neural network for hierarchical feature extraction from leaf images across 59 crop-disease combinations, an AdaBoost ensemble classifier for personalized fertilizer recommendations based on soil and crop parameters, and SHAP (SHapley Additive exPlanations) grounded in game theory to provide transparent feature attribution. By seamlessly integrating three distinct machine learning paradigms—deep learning for image analysis, ensemble methods for prediction, and Shapley values for explainability—the project demonstrates how theoretical machine learning concepts can be combined to solve real-world agricultural problems. The modular architecture enables independent optimization of each component while maintaining system cohesion. Most importantly, this project validates that responsible AI requires transparency and interpretability; explainability is not merely an academic exercise but essential for stakeholder trust and practical adoption in domains like agriculture where farmers need to understand the reasoning behind AI-driven decisions that directly impact their livelihoods, crop yields, and sustainable.

12. FUTURE SCOPE

1. Mobile and offline accessibility will enable farmers to use the system through apps, voice commands, and SMS even without internet connectivity.
2. Expanded crop and disease coverage will include more crop types, pest detection, weed identification, and regional variations.
3. Real-time IoT integration will use sensors and weather data to provide live monitoring, alerts, and historical farm insights.
4. Advanced machine learning techniques will improve accuracy using methods like transfer learning, federated learning, and reinforcement learning.
5. Sustainability and impact focus will promote eco-friendly farming, efficient resource usage, and measurable benefits for farmers.

13. REFERENCES

1. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 51–56.
2. Streamlit Documentation. (2024). Streamlit: The Fastest Way to Build Data Apps. Available at: <https://streamlit.io/>
3. Data for: Identification of Plant Leaf Diseases Using a 9-layer Deep Convolutional Neural Network
<https://data.mendeley.com/datasets/tywbtsjrjv/1>
4. Rice Leaf Disease Image Dataset
<https://data.mendeley.com/datasets/fwcj7stb8r/1>
5. Disease Dataset of Wheat: Original, Augmented, and Balanced for Deep Learning
<https://data.mendeley.com/datasets/5gc7hwydwg/1>
6. Convolutional Neural Networks
<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks/>